DSEC-I

Group-A Elective-I

BCA-3004T

BCA Semester III

Max Marks: Theory 100
(Int: 25; Ext: 75) · Practical: 100

3 Credits
(15 hrs theory + 60 hrs practical)

Feature Engineering

Lecture 1 — Introduction to Data and Features
Importance of Features in Machine Learning

90 Minutes **35 Slides**

Dept. of Computer Application · BCA III Semester · Academic Year 2025–26

Prerequisites:

Basic Python Programming · Pandas & NumPy basics · No prior ML knowledge required

Lecture Overview

WHAT WE COVER TODAY

01

What is Data?

Types of data, structured vs unstructured, real-world examples

02

What are Features?

Definition, feature vs target, role in ML pipeline

03

Data Types of Features

Numerical, categorical, ordinal, discrete, continuous, interval, ratio

04

Why Features Matter?

Garbage-in garbage-out principle, feature quality impact on models

🎯 **Key Question for Today:** If you give a bad set of features to a perfect ML algorithm — what will happen? (Answer on Slide 8!)

Quick Context — What is Machine Learning?

FOUNDATION CONCEPT

Before features — let us understand what Machine Learning is.

Traditional Programming	Machine Learning
Programmer writes explicit rules	Machine learns rules from data automatically
Rules + Data → Output	Data + Output → Rules (Model)
If-else logic for every case	Learns patterns, handles complexity
Example: if marks > 60: pass	Example: model predicts pass/fail from patterns
Works well for simple, known problems	Works well for complex, real-world problems

Feature Engineering sits right at the heart of ML — it is the art of preparing the RIGHT data for the algorithm to learn from.

The Machine Learning Pipeline — Where Does Feature Engineering Fit?

ML PIPELINE

OUR FOCUS

Raw Data

CSV files, databases, images, text, sensor readings — messy, incomplete

Cleaning

Remove nulls, fix errors, handle duplicates, correct formats

Feature Eng.

Transform raw data into meaningful numerical inputs the model can use

Training

Algorithm learns patterns — Linear Regression, Decision Tree, Neural Net etc.

Evaluation

Accuracy, Precision, Recall, F1 Score — how good is the model?

Deploy

Use model on new unseen data — real-world predictions

★ Studies show Feature Engineering contributes 60–80% of the effort in a typical ML project — and has the largest impact on model accuracy.

What is Data?

SECTION 1: DATA

Data is any collection of facts, figures, observations, or measurements that can be recorded and analysed.

Structured Data

- Organised in rows & columns
- Stored in databases / spreadsheets
- Example: Student marks table
- Example: Bank transactions
- Easy for algorithms to process

Unstructured Data

- No pre-defined format
- Hard to analyse directly
- Example: Emails, social media posts
- Example: Photos, videos, audio
- Needs special preprocessing

Semi-structured Data

- Partially organised
- Has tags or markers
- Example: XML, JSON files
- Example: HTML web pages
- Common in APIs & web data

In Machine Learning we primarily work with structured data — tables with rows (samples) and columns (features).

Data in the Real World — Examples You Already Know

REAL-WORLD CONTEXT

Student Records

Column (Feature)	Sample Value
Roll No	101
Name	Amit
Maths	78
Science	85
Result	Pass

Medical Records

Column (Feature)	Sample Value
Patient ID	P001
Age	45
BP	120/80
Sugar	Normal
Diagnosis	Healthy

E-Commerce

Column (Feature)	Sample Value
Product ID	A101
Price	499
Ratings	4.2
Reviews	230
Category	Electronics

Each row = one observation (one student / one patient / one product) | Each column = one Feature

What is a Feature?

SECTION 2: FEATURES

A **Feature** (also called an Attribute or Variable or Predictor) is one single measurable property or characteristic of the data being observed.

Feature vs. Target — Know the Difference

Term	Also Called	Example (Student Data)
Feature	Input / Attribute / Predictor / X	Maths Marks, Age, Attendance %
Target	Label / Output / Y / Dependent Variable	Pass or Fail, Final Grade

Think of it this way:

Features are the CLUES. Target is the ANSWER. ML finds the pattern from clues to predict the answer.

A Pandas DataFrame — Features Highlighted

RollNo	Age ← Feature	Maths ← Feature	Attend% ← Feature	Result ← TARGET
101	19	78	85	Pass
102	20	55	60	Fail
103	19	90	92	Pass
104	21	45	50	Fail

← Each column (except Result) is a Feature | Result is the Target →

The GIGO Principle — Why Feature Quality is Everything

GIGO = Garbage In, Garbage Out — If you feed bad features to any model, you will ALWAYS get bad predictions, no matter how powerful the algorithm.

Answer to the question from Slide 2:

A perfect algorithm with bad features → POOR model. A simple algorithm with great features → GOOD model.

Feature Quality Impact:

Scenario	Features Used	Accuracy
Raw unprocessed data	Age stored as string, missing values, no scaling	52%
After basic cleaning	Nulls filled, text converted to numbers	68%
After feature engineering	Scaled, encoded, new derived features added	89%

What Feature Engineering Actually Does

- ✓ Handles missing values so no information is lost
- ✓ Converts text/categories into numbers (algorithms only understand numbers)
- ✓ Scales features so large numbers don't dominate
- ✓ Creates new features from existing ones (e.g., BMI from height + weight)
- ✓ Removes irrelevant features that add noise
- ✓ Transforms skewed data into a normal distribution

Data Types of Features — Part 1: Numerical

SECTION 3: FEATURE DATA TYPES

Numerical features contain numbers that represent measurable quantities.

Continuous Features

Can take any value within a range — infinite possible values between two points

Feature	Example Values
Height (cm)	165.2, 172.8, 158.0
Weight (kg)	58.5, 72.1, 81.3
Temperature (°C)	36.6, 37.2, 38.1
Salary (₹)	25000.50, 45000.00

Discrete Features

Can only take specific separate (countable) values — no values in between

Feature	Example Values
Number of children	0, 1, 2, 3 (not 1.5!)
Books borrowed	0, 1, 2, 3, 4
Exam attempts	1, 2, 3
Number of subjects	5, 6, 7

Key Distinction: Continuous = measured (ruler/scale/thermometer) | Discrete = counted (whole numbers only)
Example: Height = continuous (172.5 cm is valid) | Number of siblings = discrete (2.5 siblings makes no sense!)

Python Tip: Continuous features → dtype float64 | Discrete features → dtype int64 | Check with: `df.dtypes`

Data Types of Features — Part 2: Categorical

SECTION 3: FEATURE DATA TYPES

Categorical features represent groups, labels, or names — they do NOT have mathematical meaning in their raw form.

Nominal (No Order)

Categories with no natural ranking or order

Feature	Categories
Gender	Male, Female, Other
City	Delhi, Mumbai, Chennai
Blood Group	A, B, AB, O
Course	BCA, MCA, B.Tech

Note: Cannot say Delhi > Mumbai — they are just different!

Ordinal (Has Order)

Categories WITH a natural ranking — but gap between ranks may NOT be equal

Feature	Categories
Education	School < Graduate < Post-Graduate
Satisfaction	Poor < Average < Good < Excellent
Size	S < M < L < XL
Grade	D < C < B < A

Note: We know A > B > C but we cannot say A is exactly twice as good as B

Key Rule: Machine Learning algorithms CANNOT use text/category labels directly. We must ENCODE them into numbers. This encoding is a major part of Feature Engineering (covered later).

Advanced: Interval and Ratio Scales

SECTION 3: FEATURE DATA TYPES

These are advanced measurement scales for numerical data — important in statistics and ML preprocessing.

Interval Scale

Ordered numeric values where differences are meaningful, but there is NO true zero (zero does not mean 'absence of').

Feature	Why this scale?
Temperature (°C / °F)	0°C does NOT mean no temperature — it is just freezing point
Year	Year 0 does not mean time started
IQ Score	IQ 0 does not mean no intelligence
Calendar dates	January 1 is not 'no date'

⚑ Can ADD and SUBTRACT values. CANNOT multiply/divide meaningfully. 40°C is NOT twice as hot as 20°C in absolute terms.

Ratio Scale

Ordered values with meaningful differences AND a true absolute zero (zero really means absence of the quantity).

Feature	Why this scale?
Weight (kg)	0 kg = truly no weight
Height (cm)	0 cm = truly no height
Income (₹)	₹0 = truly no income
Age (years)	0 years = just born

⚑ Can ADD, SUBTRACT, MULTIPLY and DIVIDE. 40 kg IS twice as heavy as 20 kg. Most physical measurements are ratio scale.

In Practice: For ML, what matters most is Numerical vs Categorical. Interval/Ratio distinction is mainly important when choosing statistical tests and in domain-specific preprocessing.

Summary — All Feature Data Types at a Glance

SUMMARY TABLE

Use this table as a quick reference to identify the type of any feature in a dataset.

Type	Sub-type	Order?	True Zero?	Math Operations	Example	In Python
Categorical	Nominal	No ✗	N/A	None	Gender, City, Blood Type	object / str
Categorical	Ordinal	Yes ✓	N/A	Order only	Grade (A>B>C), Size (S<M<L)	Categorical / int with order
Numerical	Discrete	Yes ✓	Yes ✓	+, −, ×, ÷	No. of children, Attempts	int64
Numerical	Continuous	Yes ✓	Yes ✓	+, −, ×, ÷	Height, Weight, Salary	float64
Numerical	Interval	Yes ✓	No ✗	+, − only	Temperature °C, IQ, Year	float64
Numerical	Ratio	Yes ✓	Yes ✓	+, −, ×, ÷	Age, Weight, Income	float64

Needs Encoding → Nominal & Ordinal categories MUST be converted to numbers before feeding to ML models.

Needs Scaling → Continuous & Ratio features with large ranges often need normalization or standardization.

Practice — Identify the Feature Type

CLASS ACTIVITY

Look at this sample dataset. For each column — identify: (1) Numerical or Categorical? (2) Sub-type?

StudentID	Age	City	Grade	Height_cm	Siblings	Temp_F	Income_₹	Result
S001	19	Delhi	A	165.5	2	98.6	25000	Pass
S002	20	Mumbai	C	172.0	0	99.1	32000	Fail
S003	19	Pune	B	158.8	1	97.8	28000	Pass

Answers:

Column	Type	Sub-type	Reasoning
StudentID	Categorical	Nominal	Just a label — no order, no math possible
Age	Numerical	Discrete	Whole years — countable, true zero (0 = just born)
City	Categorical	Nominal	No ranking: Delhi is not 'more' than Mumbai
Grade	Categorical	Ordinal	A > B > C but gap between them is not measurable
Height_cm	Numerical	Continuous / Ratio	Can be 165.5 — measured, true zero
Siblings	Numerical	Discrete	Whole numbers only, true zero
Temp_F	Numerical	Interval	0°F does not mean 'no temperature'
Income_₹	Numerical	Continuous / Ratio	₹0 = no income — true zero exists

Why Features are Important in Machine Learning — Core Reasons

SECTION 4: IMPORTANCE OF FEATURES

1 Algorithms Only Understand Numbers

ML algorithms perform mathematical operations — matrix multiplications, dot products, distances. They CANNOT process text, categories, or missing values directly. Features must be numeric and complete.

2 Irrelevant Features Add Noise

Including columns like 'Student Name' or 'Roll Number' adds no predictive value but increases computation time and can confuse the model. This is called the Curse of Dimensionality.

3 Features Determine Model Boundaries

A model can only learn from information you give it. If you forget to include 'Attendance' in a pass/fail predictor, the model can never learn that attendance affects results — no matter how powerful the algorithm.

4 Feature Scale Affects Distance Calculations

If one feature has range 0–100 (marks) and another has range 0–1,000,000 (salary), algorithms like KNN will be dominated by salary. Proper scaling ensures all features contribute equally.

Understanding Feature Importance — A Simple Analogy

INTUITION BUILDER

Think of predicting if a student will pass or fail — what information would a human expert use?

✓ Useful Features (Use These)

- **Marks in previous subjects** (Direct indicator of academic performance)
- **Attendance percentage** (Regular attendance → more learning)
- **Hours of study per day** (Effort metric)
- **Number of assignments submitted** (Engagement indicator)
- **Class participation score** (Active learning signal)

✗ Useless/Harmful Features (Avoid)

- ✗ **Student Name** (Names don't predict results — adds noise)
- ✗ **Roll Number** (Just an ID — no predictive value)
- ✗ **Phone Number** (Completely irrelevant to pass/fail)
- ✗ **Address** (Where you live does not determine results)
- ✗ **Aadhaar Number** (Personal ID — not a learning signal)

Feature Engineering is about selecting and creating the RIGHT features — the ones that give the model maximum useful signal.

Real Dataset Walkthrough — Student Performance Dataset

HANDS-ON EXAMPLE

Let us look at a real-world style dataset and analyse every feature.

student_id	age	gender	city	prev_marks	attendance_pct	study_hrs	grade	result
S001	19	M	Delhi	72	85.0	4.5	B	Pass
S002	20	F	Mumbai	45	60.0	2.0	D	Fail
S003	21	M	Pune	88	92.0	6.0	A	Pass
S004	19	F	Delhi	55	NaN	3.5	C	Fail
S005	22	M	Chennai	91	95.0	7.0	A	Pass

Feature Analysis Table:

Column	Feature/Target	Type	Issue?	FE Action Needed
student_id	❌ Useless	Nominal	None	DROP — just an ID, no predictive value
age	Feature	Discrete (Ratio)	None	Ready to use — may normalize
gender	Feature	Nominal Categorical	None	ENCODE → One-Hot Encoding
city	Feature	Nominal Categorical	None	ENCODE → One-Hot or Target Encoding
prev_marks	Feature	Continuous (Ratio)	None	Normalize / Scale (0–100 range)
attendance_pct	Feature	Continuous (Ratio)	NaN in row 4!	Fill missing value, then scale
study_hrs	Feature	Continuous (Ratio)	None	Ready after scaling
grade	Feature	Ordinal Categorical	None	ENCODE → Ordinal Encoding (D=0,C=1,B=2,A=3)
result	🎯 TARGET	Nominal Binary	None	ENCODE → 0=Fail, 1=Pass

Deep Dive — Features (X) vs. Target (y) in Code

CODE WALKTHROUGH

In Python/Pandas, we always separate features (X) from the target (y) before training a model.

```
import pandas as pd

# Load the dataset
df = pd.read_csv('student_data.csv')

# Separate Features (X) and Target (y)
X = df.drop(columns=['result'])   # All columns EXCEPT result
y = df['result']                   # Only the target column

# Check shapes
print(X.shape)    # (5, 8)  → 5 rows, 8 features
print(y.shape)    # (5,)    → 5 target values

# View feature names
print(X.columns.tolist())
# ['age','gender','city','prev_marks','attendance_pct','study_hrs','grade']
```

What This Means

X (Features):

The input data — everything the model learns FROM. Shape = (rows, columns).

y (Target):

What we want to predict. A single column. Shape = (rows,).

df.drop():

Remove the target column from features to avoid data leakage.

X.shape:

Tells us how many samples and features we have.

X.columns:

Lists all feature names — useful to verify correct columns selected.

Common Feature Issue — Missing Values (NaN)

DATA QUALITY

Real-world data is almost always incomplete. Missing values are one of the most common problems in Feature Engineering.

Why Missing Values Occur

- Survey questions skipped by user
- Sensor malfunction / equipment error
- Data entry error or forgotten field
- Feature not applicable (e.g. no spouse for single person)
- Data from different sources merged
- System crash during data collection

How to Handle Missing Values (Preview)

Method	When to Use	Python Code
Drop rows with NaN	Very few rows missing (< 5%)	df.dropna()
Fill with Mean	Numerical, no extreme outliers	df['col'].fillna(df['col'].mean())
Fill with Median	Numerical, outliers present	df['col'].fillna(df['col'].median())
Fill with Mode	Categorical features	df['col'].fillna(df['col'].mode()[0])
Fill with constant	Domain-specific known default	df['col'].fillna(0) or fillna('Unknown')
Forward Fill	Time-series data	df.fillna(method='ffill')

Lab Programme 1: Handle missing values by filling with mean/median/mode. Lab Programme 2: Drop rows with too many missing values.

Why Feature Scaling is Essential

PREPROCESSING CONCEPT

Feature Scaling ensures all numerical features are on the same scale — so no single feature dominates the model.

The Problem — Without Scaling

Feature	Range	Problem
Salary (₹)	10,000 – 1,00,000	DOMINATES — huge numbers
Age (years)	18 – 60	Tiny compared to salary
Marks (%)	0 – 100	Tiny compared to salary

The Solution — After Scaling

Feature	After Min-Max Scale	Result
Salary (₹)	0.0 – 1.0	Equal weight ✓
Age (years)	0.0 – 1.0	Equal weight ✓
Marks (%)	0.0 – 1.0	Equal weight ✓

Min-Max Normalization

`Formula: X_scaled = (X - X_min) / (X_max - X_min)`

Result range: 0.0 to 1.0 | Use when: data has no extreme outliers

Standardization (Z-Score)

`Formula: X_scaled = (X - mean) / std_dev`

Result: mean=0, std=1 | Use when: data has outliers or is normally distributed

Lab Programme 3: Scale numerical features using Min-Max normalization for a dataset with columns 'Height' and 'Weight'.

Feature Engineering — The Complete Process

PROCESS OVERVIEW

01 Understand the Data

- Load dataset
- df.info(), df.describe()
- Identify feature types
- Spot missing values

02 Clean the Data

- Handle missing values
- Remove duplicates
- Fix incorrect entries
- Standardise formats

03 Transform Features

- Encode categoricals
- Scale numericals
- Handle outliers
- Log-transform skewed

04 Create New Features

- Combine existing ones
- Extract from dates
- Bin continuous data
- Domain knowledge

05 Select Features

- Drop irrelevant cols
- Check correlations
- Filter / Wrapper
- Reduce dimensions (PCA)

This course (DSEC-I) covers Steps 1–3 in depth, with an introduction to Steps 4 and 5. Steps 4–5 are covered further in ML courses.

Exploratory Data Analysis (EDA) — Understanding Your Data First

EDA PREVIEW

Before any feature engineering, we MUST explore the data. Lab Programme 4: EDA with histograms and box plots.

`df.shape`

How many rows and columns do we have?

`df.info()`

Data types of each column, non-null counts

`df.describe()`

Count, mean, std, min, max, quartiles for numericals

`df.isnull().sum()`

Count of missing values per column

`df['col'].value_counts()`

Frequency of each category in categorical features

`df.hist()`

Histogram — distribution shape of numerical features

`df.corr()` → Correlation Matrix — shows how strongly each feature relates to every other feature. Values close to +1 or -1 = strong relationship. Close to 0 = weak. Lab Programme 5 covers this.

Correlation Matrix — Lab Programme 5 Explained

CORRELATION ANALYSIS

A correlation matrix shows the relationship strength between every pair of numerical features. Range: −1 to +1.

	age	prev_marks	attendance	study_hrs
age	1.00	0.05	-0.12	0.08
prev_marks	0.05	1.00	0.72	0.68
attendance	-0.12	0.72	1.00	0.61
study_hrs	0.08	0.68	0.61	1.00

Reading the Matrix

0.72 **prev_marks ↔ attendance** Strong positive — attend class, score well

0.68 **prev_marks ↔ study_hrs** Strong positive — study more, score more

0.05 **age ↔ prev_marks** Near zero — age does NOT predict marks

-0.12 **age ↔ attendance** Slight negative — older slightly less attendance

Correlation Values — Interpretation Guide

Range	Interpretation	Action
0.9 to 1.0	Very Strong Positive	Consider removing one — highly redundant
0.7 to 0.9	Strong Positive	Both likely useful but watch for multicollinearity
0.5 to 0.7	Moderate Positive	Both useful — keep both
0.0 to 0.3	Weak / No relation	Feature may not be predictive
Negative	Inverse relationship	As one increases, other decreases

Feature Transformation — Binning Continuous Data

TRANSFORMATION TECHNIQUE

Binning (Discretisation) converts a continuous numerical feature into categorical ranges (bins). Lab Programme 6.

Before Binning — Raw Marks

StudentID	Marks
S001	78
S002	45
S003	91
S004	55
S005	33

After Binning — Marks as Grade Bins

StudentID	Marks	Grade_Bin
S001	78	Good (60-80)
S002	45	Average (40-60)
S003	91	Excellent (80-100)
S004	55	Average (40-60)
S005	33	Poor (0-40)

Python Code:

```
bins   = [0, 40, 60, 80, 100]
labels = ['Poor', 'Average', 'Good', 'Excellent']

df['Grade_Bin'] = pd.cut(df['Marks'],
                         bins=bins,
                         labels=labels)
```

Why Binning Helps

- ✓ Reduces effect of small measurement errors
- ✓ Makes continuous features more interpretable
- ✓ Captures non-linear relationships
- ✓ Useful when exact value matters less than range

Encoding Categorical Features — Converting Text to Numbers

ENCODING TECHNIQUES

ML algorithms require numerical input. We must encode all categorical features before training.

One-Hot Encoding — for Nominal Data

Creates a new binary (0/1) column for each category. Use when categories have NO order.

City	City_Delhi	City_Mumbai	City_Pune
Delhi	1	0	0
Mumbai	0	1	0
Pune	0	0	1
Delhi	1	0	0

```
pd.get_dummies(df['City'])  or  OneHotEncoder()
```

Label / Ordinal Encoding — for Ordinal Data

Assigns an integer to each category maintaining their natural order. Use ONLY when categories have order.

Grade (Original)	Encoded Value	Reasoning
D (Fail)	0	Lowest
C	1	Low
B	2	Medium
A	3	High
A+	4	Highest

```
from sklearn.preprocessing import OrdinalEncoder
```

⚠ **Warning:** NEVER use Label Encoding on Nominal data (e.g. encoding Delhi=1, Mumbai=2) — the model will think Mumbai > Delhi which is meaningless and will cause wrong predictions!

Course Outcomes — How This Lecture Maps to CO1 and CO2

COURSE OUTCOMES

CO1 — Understand Feature Engineering in ML Workflow

Covered in this lecture:

✓ What is Machine Learning — Slide 3

✓ The ML Pipeline and where FE fits — Slide 4

✓ GIGO Principle — quality of features matters — Slide 8

✓ Why Features are Important in ML — Slide 14

✓ Feature vs. Target (X and y) — Slides 7, 17

Coming in future lectures:

○ Advanced FE for model improvement (later sessions)

○ Cross-validation and feature selection metrics

CO2 — Apply Data Preprocessing — Handle Missing, Scale, Clean

Covered in this lecture:

✓ Types of features and data types — Slides 9–12

✓ Identifying feature types in real datasets — Slide 13

✓ Missing value detection and handling — Slide 18

✓ Feature scaling — Min-Max and Z-Score — Slide 19

✓ EDA — explore before you transform — Slides 21–22

Coming in future lectures:

○ Advanced imputation (KNN, iterative) — later lecture

○ Handling outliers with IQR and Z-score — Lecture 2

CO3 (encoding from text/image/time-series) and CO4 (feature selection + model performance) are covered in Lectures 2 and 3.

Lab Programme 1 — Handle Missing Values (Mean/Median/Mode)

LAB PROGRAMME 1

Problem: Fill missing values in Age (numerical) and City (categorical) columns using appropriate strategies.

```
import pandas as pd
import numpy as np

# Sample Dataset with Missing Values
data = {
    'Name':  ['Amit', 'Priya', 'Raj', 'Sita', 'Mohan'],
    'Age':   [19, np.nan, 21, 20, np.nan],     # 2 missing
    'City':  ['Delhi', 'Mumbai', np.nan, 'Pune', np.nan],  # 2 missing
    'Marks': [78, 55, 88, 45, 91]
}
df = pd.DataFrame(data)

# Step 1: Check missing values
print(df.isnull().sum())

# Step 2: Fill numerical 'Age' with MEAN
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Step 3: Fill categorical 'City' with MODE
df['City'] = df['City'].fillna(df['City'].mode()[0])

print(df)  # Verify — no NaN values remain
```

Lab Programme 3 — Min-Max Normalization

LAB PROGRAMME 3

Problem: Scale Height and Weight columns to the range [0, 1] using Min-Max normalization.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = {
    'Name':   ['Amit','Priya','Raj','Sita','Mohan'],
    'Height': [165, 172, 158, 180, 155],   # in cm
    'Weight': [58, 72, 65, 85, 52]          # in kg
}
df = pd.DataFrame(data)

# Apply Min-Max Scaling
scaler = MinMaxScaler()
df[['Height_scaled','Weight_scaled']] = \
    scaler.fit_transform(df[['Height','Weight']])

print(df[['Name','Height','Height_scaled',
          'Weight','Weight_scaled']])
```

Output DataFrame

Name	Height	H_scaled	Weight	W_scaled
Amit	165	0.40	58	0.18
Priya	172	0.68	72	0.60
Raj	158	0.12	65	0.39
Sita	180	1.00	85	1.00
Mohan	155	0.00	52	0.00

Formula applied:

```
(X - X_min) / (X_max - X_min)
```

Min=0, Max=1 after scaling
Sita (tallest, heaviest) → 1.0
Mohan (shortest, lightest) → 0.0

Python Tools for Feature Engineering — Quick Reference

TOOLS & LIBRARIES

You will use these Python libraries throughout this course. Understanding their role is essential.

pandas

Data manipulation — load, clean, filter, group, merge DataFrames

```
Key functions: pd.read_csv()  |  df.fillna()  |  df.drop()  |
pd.get_dummies()  |  df.groupby()
```

numpy

Numerical computing — array operations, mathematical functions

```
Key functions: np.mean()  |  np.median()  |  np.std()  |  np.log()  |
np.where()
```

sklearn.preprocessing

Scaling, encoding — ML-ready transformations

```
Key functions: MinMaxScaler  |  StandardScaler  |  OneHotEncoder  |
OrdinalEncoder  |  LabelEncoder
```

matplotlib / seaborn

Visualisation — plots to understand data distribution

```
Key functions: plt.hist()  |  sns.boxplot()  |  sns.heatmap()  |
plt.scatter()  |  sns.pairplot()
```

Common Mistakes in Feature Engineering — Avoid These!

PITFALLS TO AVOID

#1 Using ID Columns as Features

✗ Including 'student_id', 'roll_no' in X

⚠ **Impact:** Model memorises IDs — zero generalisation

✓ **Fix:** Always drop ID/index columns before training

#2 Label Encoding Nominal Data

✗ City: Delhi=0, Mumbai=1, Pune=2

⚠ **Impact:** Model thinks Pune > Mumbai > Delhi — nonsense!

✓ **Fix:** Use One-Hot Encoding for nominal categories

#3 Scaling BEFORE Splitting Train/Test

✗ Fit scaler on full dataset then split

⚠ **Impact:** Data leakage — test data influences scaling

✓ **Fix:** Split first, then fit scaler on train only

#4 Forgetting to Handle Missing Values

✗ Passing df with NaN to sklearn

⚠ **Impact:** ValueError — most algorithms reject NaN input

✓ **Fix:** Always check df.isnull().sum() before training

#5 Including Target in Feature Matrix X

✗ Not dropping 'result' from X

⚠ **Impact:** Model 'cheats' by using the answer as input

✓ **Fix:** X = df.drop(columns=['result']) explicitly

#6 Using Categorical Strings Directly

✗ Passing 'Male'/'Female' as text to model

⚠ **Impact:** TypeError — strings cannot be used in math

✓ **Fix:** Encode all string columns to numbers first

Feature Engineering Checklist — Before You Train Any Model

CHECKLIST

Use this checklist every time you prepare a dataset for machine learning.

Understand

- ✓ df.shape — know how many rows and columns
- ✓ df.info() — data types, null counts
- ✓ df.describe() — stats for numericals
- ✓ Identify feature types (numerical / categorical / ordinal)

Clean

- ✓ Check df.isnull().sum() — handle all NaN values
- ✓ Remove duplicate rows: df.drop_duplicates()
- ✓ Fix data type mismatches (e.g. age stored as string)
- ✓ Drop irrelevant columns (IDs, names)

Transform

- ✓ Encode all categorical columns (OHE or Ordinal)
- ✓ Scale all numerical columns (MinMax or Standard)
- ✓ Handle outliers if any (IQR or Z-score check)
- ✓ Log-transform skewed features if needed

Verify

- ✓ X = df.drop(columns=['target']) — features only
- ✓ y = df['target'] — target only
- ✓ X.isnull().sum() == 0 — no missing values left
- ✓ X.dtypes — all columns must be numerical (float/int)

Quick Recap — What We Learned Today

LECTURE RECAP

01 **Data Types**

Structured, Unstructured, Semi-structured. ML works mostly with structured (tabular) data.

02 **Feature vs. Target**

Features (X) = inputs, Target (y) = what we predict. Always separate them.

03 **Feature Data Types**

Numerical (discrete/continuous, interval/ratio) and Categorical (nominal/ordinal).

04 **GIGO Principle**

Feature quality is more important than algorithm choice. Bad features = bad model.

05 **Missing Values**

Handle with mean (numerical), median (with outliers), or mode (categorical).

06 **Encoding**

Nominal → One-Hot Encoding. Ordinal → Label/Ordinal Encoding.

07 **Scaling**

Min-Max Normalization (0 to 1). Standardization (mean=0, std=1).

08 **EDA First**

Always explore data before transforming. Use df.info(), describe(), hist(), corr().

Lab Programmes — Covered / Referenced in This Lecture

LAB SUMMARY

All 21 lab programmes will be covered across lectures. These are the ones relevant to today's content.

Lab #	Programme	Key Concepts Used	Python Functions	Status
Lab 1	Handle missing values — mean/median/mode	Missing values, feature types, NaN	fillna(), mean(), median(), mode()	✓ This Lecture
Lab 2	Clean dataset — remove invalid entries	Data cleaning, df.drop()	dropna(), drop_duplicates(), isnull()	✓ This Lecture
Lab 3	Min-Max normalization — Height, Weight	Feature scaling, ratio scale	MinMaxScaler, fit_transform()	✓ This Lecture
Lab 4	EDA — histogram and box plots	Distribution analysis, outlier detection	df.hist(), sns.boxplot(), plt.show()	✓ This Lecture
Lab 5	Correlation matrix visualisation	Feature correlation, heatmap	df.corr(), sns.heatmap()	✓ This Lecture
Lab 6	Binning continuous data	Discretisation, pd.cut()	pd.cut(), pd.qcut(), bins, labels	✓ This Lecture
Lab 7	Polynomial & interaction features	Feature creation, numerical transforms	PolynomialFeatures, np operations	○ Lecture 2
Lab 9	One-hot encoding — categorical features	Nominal encoding, dummy variables	pd.get_dummies(), OneHotEncoder()	✓ Introduced today

All lab programmes follow the same datasets used in examples. Students should run each code block and verify the output matches the table shown.

Self-Assessment — Test Your Understanding

SELF ASSESSMENT

Answer these questions to check your understanding of today's lecture. No calculators needed!

1. A dataset has columns: Name, Roll_No, CGPA, Department, Attendance%, Result. Which columns should be DROPPED before training a model?

Answer: Name and Roll_No — they are IDs with no predictive value.

2. 'Size' column has values: S, M, L, XL. What type of feature is it, and what encoding should you use?

Answer: Ordinal Categorical. Use Ordinal Encoding (S=0, M=1, L=2, XL=3).

3. 'CGPA' ranges from 0 to 10, but 'Income' ranges from 10,000 to 2,00,000. What problem can this cause, and what is the fix?

Answer: Income dominates distance calculations. Fix: Apply Min-Max Scaling or Standardization.

4. df.isnull().sum() shows: Name=0, Age=3, City=2. What strategy should you use for each?

Answer: Age: fill with mean (numerical). City: fill with mode (categorical).

5. What does GIGO stand for and what does it mean for Feature Engineering?

Answer: Garbage In Garbage Out — bad input features produce bad model predictions regardless of algorithm used.

Score yourself: 5/5 = Excellent | 4/5 = Good | 3/5 = Review Slides 8–12 | <3 = Attend doubt session

Next Lecture — Feature Engineering Techniques (Lecture 2)

COMING NEXT

In the next lecture (Unit II) we will build on today's foundation and explore deeper transformation techniques.

01 Numerical Techniques

- Binning & Discretisation (deep dive)
- Polynomial Features — x^2, $x \cdot y$
- Interaction Features
- Log transformation for skewed data

02 Categorical Techniques

- One-Hot Encoding (detail + pitfalls)
- Label Encoding vs Ordinal Encoding
- Target Encoding introduction
- Handling high-cardinality categoricals

03 Feature Reduction

- Why too many features is bad
- Principal Component Analysis (PCA) intro
- Filter methods — correlation-based
- Wrapper methods — forward selection

04 Lab Programmes 7–9

- Lab 7: Polynomial & interaction features
- Lab 8: Log transformation
- Lab 9: One-hot encoding in detail
- Lab 9+: Combine all encoding types

Preparation for Next Class: Complete Lab Programmes 1–6 on your own. Bring a dataset of your choice (any CSV from Kaggle) and we will explore it together using the checklist from Slide 30.

DSEC-I

Feature Engineering

BCA Semester III

Dept. of Computer Application

Thank You

Lecture 1 — Introduction to Data and Features

5	4	6	35
Data Types	Feature Types	Lab Progs	Slides

Key Takeaway:

"Feature Engineering is not just data preparation — it is the science of giving your model the best possible chance to learn."